

TurnosRed — Informe técnico de la Actividad 4

Diseño del módulo Pacientes y Turnos, Postman con `{{baseUrl}}`, README integral y control del repositorio · Integraciones Web

Repositorio: github.com/Luismcbo/turnos-red · **Stack:** Node.js + TypeScript + Express · **Autor:** Luis Valenzuela · **Fecha:** 23/09/2026

1. Resumen y cumplimiento de la consigna

La actividad es de **diseño y documentación**: se diseñó el módulo de Pacientes y Turnos (interfaces TypeScript primero, endpoints propuestos después), se normalizó la colección de Postman con la variable `baseUrl`, se documentó toda la API en el README a partir del código real y se dejó el repositorio listo para entregar. **Los endpoints de Pacientes son una propuesta**: no están implementados y la API existente no cambió.

Requisito de la consigna	Entregable	Dónde
1. Mockup de Paciente y Turno (interfaces TS primero)	Interfaces <i>Paciente</i> , <i>TurnoMedico</i> y contratos de respuesta; solo tipos, verificadas con <i>tsc</i>	<code>src/models/</code>
2. Explicación del modelado + 2 endpoints RESTful	<i>POST /pacientes</i> y <i>GET /pacientes/:id/turnos</i> con bloques typescript y json (request, 2xx y errores)	<code>pacientes-turnos.md</code>
3. Postman con <i>baseUrl</i> y sin URLs fijas; re-ejecutar	71 requests con <code>{{baseUrl}}</code> ; corrida completa 71 requests / 314 assertions, 0 fallas	<code>postman/</code>
4. README completo generado leyendo el código	Cada endpoint (método, path, parámetros, respuestas), instalación, variables de entorno y árbol de carpetas	<code>README.md</code>
5. <code>.gitignore</code> (sin <code>node_modules</code> , <code>dist</code> , <code>.env</code> ; con <code>.env.example</code>)	Reglas verificadas con <i>git check-ignore</i>	<code>.gitignore</code>
6. Verificar contenido del repo	Versionados: <code>src/</code> completo (42 archivos), <code>src/data/</code> , <code>package.json</code> , <code>pacientes-turnos.md</code> , <code>README</code> , <code>.env.example</code>	<code>git ls-files</code>
7. Commits separados por etapa	5 commits que continúan el historial existente (sección 2)	<code>git log</code>

2. Historial de commits (Actividad 4)

Commit	Mensaje
865bb4d	docs: mockup del modulo Pacientes y Turnos Medicos (pacientes-turnos.md)
9498525	test: usar <code>{{baseUrl}}</code> en toda la coleccion de Postman (requests y ejemplos)
e241630	docs: README integral de la API (instalacion, endpoints, errores, Postman)
e49c3c9	chore: reforzar <code>.gitignore</code> (<code>.env.*</code> ignorados, <code>.env.example</code> versionado)
d06ec47	docs: capturas de evidencia de la Actividad 4

3. Modelado: Paciente y Turno médico

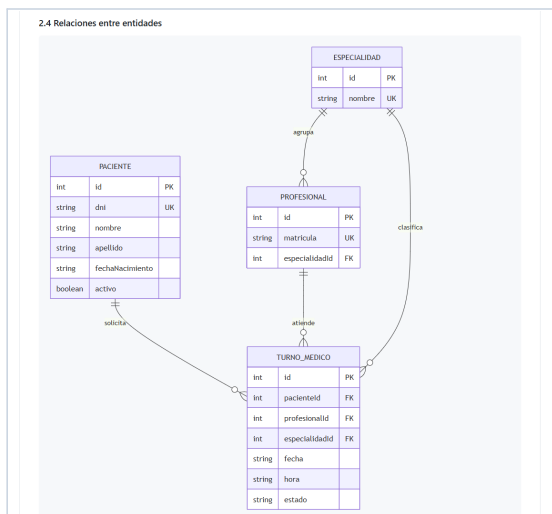
Se definió primero el contrato en TypeScript y luego se derivaron los endpoints. Decisiones principales: el turno referencia por **id** al paciente, al profesional y a la especialidad (en lugar de repetir nombre y documento como hace el `/turnos` actual); el DNI es **texto único** y el *id* numérico es la clave interna; la edad se **calcula**; la baja es **lógica** (*activo*) para conservar el historial; y el listado de turnos devuelve solo un **resumen** del paciente porque el contacto y la obra social son datos sensibles (Ley 25.326).

```
export interface Paciente {
  id: number;
  dni: string; // 7-8 digitos, unico
  nombre: string;
  apellido: string;
  fechaNacimiento: string; // ISO YYYY-MM-DD
  sexo?: 'F' | 'M' | 'X';
  contacto: { telefono: string; email?: string;
    direccion?: string };
  obraSocial?: { nombre: string;
    numeroAfiliado: string };
  activo: boolean; // baja logica
  creadoEn: string;
  actualizadoEn: string;
}
```

```
export type EstadoTurno = 'PENDIENTE' | 'CONFIRMADO'
  | 'ATENDIDO' | 'CANCELADO' | 'AUSENTE';
```

```
export interface TurnoMedico {
  id: number;
  pacienteId: number; // FK Paciente
  profesionalId: number; // FK Profesional
  especialidadId: number; // FK Especialidad
  fecha: string; // YYYY-MM-DD
  hora: string; // HH:mm
  duracionMinutos: number;
  estado: EstadoTurno;
  motivo?: string;
  observaciones?: string;
  creadoEn: string;
  actualizadoEn: string;
}
```

```
export interface TurnosDePacienteResponse {
  paciente: PacienteResumen;
  total: number;
  turnos: TurnoDePaciente[];
}
```



Relación	Regla de integridad
Paciente → Turno (1 a N)	El paciente debe existir y estar activo. Con turnos asociados se da de baja lógicamente, no se borra.
Profesional → Turno (1 a N)	Debe existir y estar activo; no puede tener dos turnos que se solapen el mismo día.
Especialidad → Turno	<i>turno.especialidadId</i> debe coincidir con <i>profesional.especialidadId</i> .
Especialidad → Profesional	Ya implementada (Actividad 3): no se elimina una especialidad con profesionales.
Turno actual → Turno médico	<i>/turnos</i> sigue funcionando sin cambios; <i>confirmado</i> (booleano) evoluciona a <i>estado</i> con 5 valores.

Captura 03: diagrama entidad-relación (Mermaid) renderizado desde *pacientes-turnos.md*.

4. Endpoints propuestos

Ambos siguen las convenciones de la API: *routes* → *controllers* → *services* → *data*, métodos *async* con *status* dinámico, validación previa con retorno anticipado y el formato de error estándar { *status*, *message*, *code*, *details* }.

4.1 POST /pacientes → 201 Created

```
POST /pacientes
{
  "dni": "31.654.210",
  "nombre": "Carlos",
  "apellido": "Ruiz",
  "fechaNacimiento": "22/03/1985",
  "sexo": "M",
  "contacto": {
    "telefono": "+54 351 555 0101",
    "email": "carlos.ruiz@example.com"
  },
  "obraSocial": {
    "nombre": "OSDE",
    "numeroAfiliado": "12345678901"
  }
}
```

```
201 Created Location: /pacientes/1
{
  "id": 1,
  "dni": "31654210", // normalizado
  "nombre": "Carlos",
  "apellido": "Ruiz",
  "fechaNacimiento": "1985-03-22",
  "sexo": "M",
  "contacto": { ... },
  "obraSocial": { ... },
  "activo": true,
  "creadoEn": "2026-09-25T14:30:00.000Z",
  "actualizadoEn": "2026-09-25T14:30:00.000Z"
}
```

```
400 VALIDATION_ERROR (campos faltantes/invalidos,
  campo desconocido, DNI duplicado)
400 INVALID_JSON | 500 INTERNAL_ERROR
```

El cliente no envía *id*, *creadoEn* ni *actualizadoEn*. El DNI acepta puntos y se normaliza; la fecha acepta YYYY-MM-DD o DD/MM/YYYY y se guarda en ISO.

4.2 GET /pacientes/:id/turnos → 200 OK

```
GET /pacientes/1/turnos?desde=2026-10-01 200 OK
{
  "paciente": { "id": 1, "dni": "31654210",
                "nombre": "Carlos", "apellido": "Ruiz" },
  "total": 2,
  "turnos": [
    {
      "id": 12, "fecha": "2026-10-05", "hora": "09:30",
      "duracionMinutos": 30, "estado": "CONFIRMADO",
      "motivo": "Control anual",
      "profesional": { "id": 4, "nombre": "Ricardo",
                      "apellido": "Paz", "matricula": "PR-1004" },
      "especialidad": { "id": 1, "nombre": "Clínica médica" }
    },
    ...
  ]
}
```

Path: id (entero positivo)
Query (opcionales, AND):
 estado PENDIENTE|CONFIRMADO|ATENDIDO|
 CANCELADO|AUSENTE
 desde YYYY-MM-DD | DD/MM/YYYY
 hasta idem (no anterior a desde)
 orden asc (defecto) | desc

Errores:
 400 VALIDATION_ERROR id no numerico,
 estado desconocido, fechas invalidas
 404 NOT_FOUND paciente inexistente
 Lista vacia => 200 con total: 0

El turno ya trae el profesional y la especialidad resueltos, para que el Frontend no haga requests adicionales. Los 8 ejemplos JSON del documento se validaron contra las interfaces con *tsc*. El documento incluye además el plan de archivos (routes, controller, service, validators) y una guía para Frontend con un tipo *ApiError* y un ejemplo con *fetch*.

3.2 POST /pacientes

Registra un paciente nuevo.

Body

Campo	Tipo	Obligatorio	Regla
id	string	SI	7 u 8 dígitos (se aceptan puntos y se normalizan: 31.654.210 → 31654210). Único.
nombre, apellido	string	SI	No vacíos, hasta 60 caracteres (se recortan espacios).
fechaNacimiento	string	SI	YYYY-MM-DD o DD/MM/YYYY, fecha real, pasada y de hasta 120 años atrás. Se guarda en ISO.
sexo	string	No	F, M o X.
contacto.telefono	string	SI	6 a 20 caracteres: dígitos, espacios, +, -, (,), .
contacto.email	string	No	Formato de correo válido, hasta 120 caracteres.
contacto.direccion	string	No	Hasta 200 caracteres.
obraSocial	objeto	No	Si se envía: nombre y numeroAfilado obligatorios (no vacíos).
activo	boolean	No	Por defecto true.

Campos no listados se rechazan ("Campo no permitido"). El cliente no envía id, createdAt ni actualizado.

Ejemplo de request

```
POST /pacientes
Content-Type: application/json

{
  "dni": "31.654.210",
  "nombre": "Carlos",
  "apellido": "Ruiz",
  "fechaNacimiento": "2026-10-05",
  "sexo": "M",
  "contacto.telefono": "011-1234-5678",
  "contacto.email": "carlos.ruiz@ejemplo.com",
  "contacto.direccion": "Calle Falsa 123",
  "obraSocial": { "nombre": "OS 123", "numeroAfilado": "456789" },
  "activo": true
}
```

3.3 GET /pacientes/:id/turnos

Lista los turnos de un paciente (histórico y próximos), con el profesional y la especialidad ya resueltos para que el Frontend no tenga que hacer requests adicionales.

Path param

Param	Tipo	Regla
id	integer	Entero positivo, id del paciente

Query params (todos opcionales, se combinan con AND)

Param	Valores	Descripción
estado	PENDIENTE, CONFIRMADO, ATENDIDO, CANCELADO, AUSENTE	Solo turnos en ese estado (no distingue mayúsculas)
desde	YYYY-MM-DD o DD/MM/YYYY	Fecha mínima, inclusive
hasta	YYYY-MM-DD o DD/MM/YYYY	Fecha máxima, inclusive (no puede ser anterior a desde)
orden	asc (por defecto), desc	Orden por fecha y hora

Ejemplo de request

```
GET /pacientes/1/turnos?desde=2026-10-01&orden=asc
```

Respuesta 200 OK

```
{
  "paciente": {
    "id": 1,
    "dni": "31654210",
    "nombre": "Carlos",
    "apellido": "Ruiz"
  },
  "total": 2,
  "turnos": [
    {
      "id": 12, "fecha": "2026-10-05", "hora": "09:30",
      "duracionMinutos": 30, "estado": "CONFIRMADO",
      "motivo": "Control anual",
      "profesional": { "id": 4, "nombre": "Ricardo", "apellido": "Paz", "matricula": "PR-1004" },
      "especialidad": { "id": 1, "nombre": "Clínica médica" }
    },
    ...
  ]
}
```

Capturas 04 y 05: secciones de *POST /pacientes* y *GET /pacientes/:id/turnos* en *pacientes-turnos.md* (render local del Markdown).

5. Postman: {{baseUrl}} en toda la colección

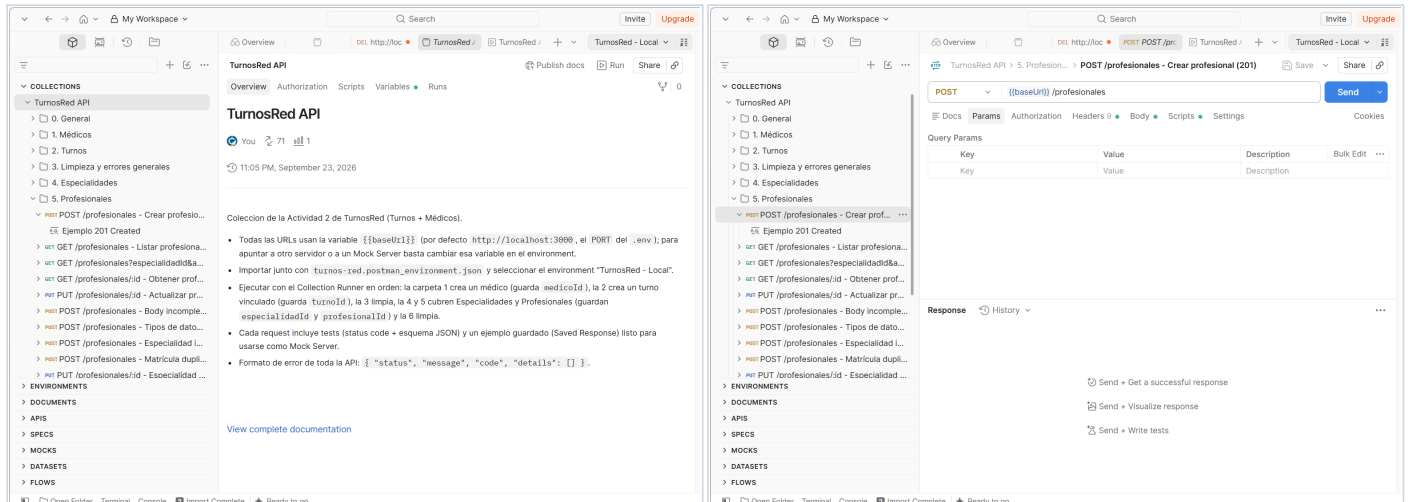
Auditoría. Las 71 requests ya usaban {{baseUrl}}; las URLs fijas estaban en los **ejemplos guardados** (Saved Responses), y se corrigieron todos. baseUrl = http://localhost:3000 queda definida como variable de la colección y en el environment postman/turnos-red.postman_environment.json. La colección se sigue exportando como turnos-red.postman_collection.json.

Prueba de que ninguna URL queda fija (Newman)

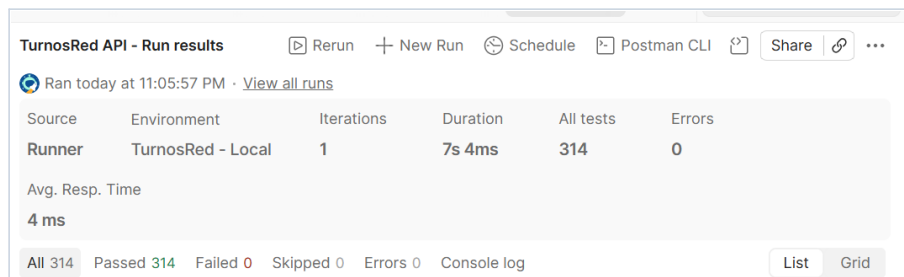
Corrida	Destino efectivo	Requests OK	Assertions OK	Resultado esperado
A. environment con baseUrl=:3000	localhost:3000	71/71	314/314	Todo en verde
B. servidor en :3005 y baseUrl=:3005	localhost:3005	71/71	314/314	Todo en verde (cambia solo la variable)
C. baseUrl=:3999 (nada escucha)	localhost:3999	0/71	0/224	Todo falla: no hay URL fija que llegue al :3000
D. sin environment (variable de colección)	localhost:3000	71/71	314/314	Todo en verde

En C las assertions que llegan a ejecutarse son 224 (las que dependen de ids encadenados no corren) y fallan todas; lo relevante es que las **71 requests** fallan al apuntar a un puerto vacío.

Evidencia en Postman de escritorio



Capturas 10 y 12: colección y environment importados (71 requests, environment “TurnosRed - Local”) y la request `POST {{baseUrl}}/profesionales` con la variable en la URL.

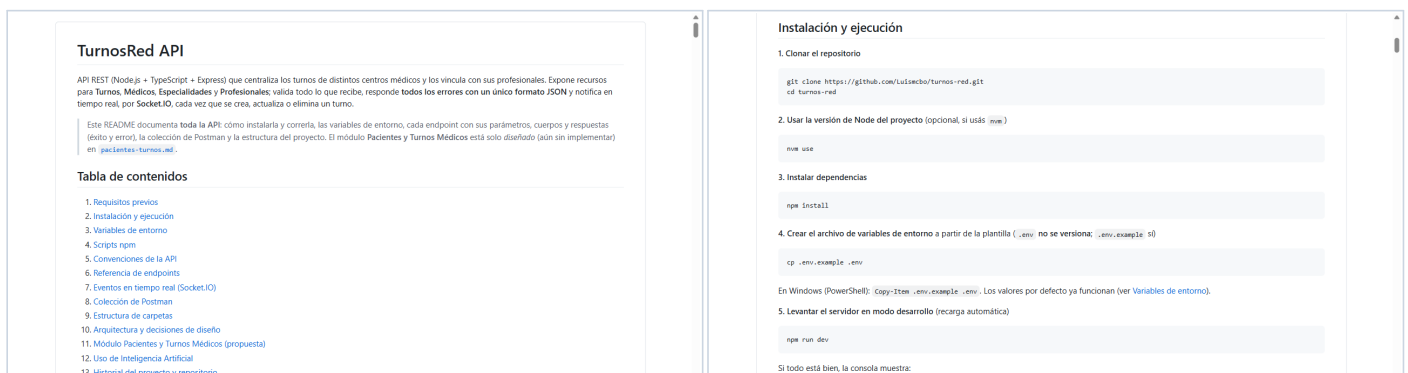


Captura 11: Collection Runner con el environment “TurnosRed - Local”: 314 tests, 314 passed, 0 failed, 0 errores (71 requests, 1 iteración).

6. README integral

El README se **generó leyendo el código real** (routes, controllers, schemas y validators) y las respuestas que se muestran son las que devolvió el servidor al ejecutar cada endpoint, no texto escrito de memoria. Contiene:

- **Endpoints (5 recursos, 21 en total):** General, Turnos, Médicos, Especialidades y Profesionales, cada uno con método y path, descripción, parámetros de ruta, query y body (tipo, obligatoriedad, regla) y respuestas de éxito y de error con JSON real.
- **Instalación y ejecución paso a paso** (requisitos, clonar, *npm install*, crear *.env* desde *.env.example*, *npm run dev*, build, producción, lint/format) con la salida esperada en consola.
- **Tabla de variables de entorno** (*PORT*, *TURNOS_DATA_PATH*, *MEDICOS_DATA_PATH*: descripción, valor por defecto), convenciones, códigos de estado y formato de error estándar.
- **Estructura de carpetas**, arquitectura y decisiones, Socket.IO, uso de la colección de Postman con *baseUrl*, propuesta de Pacientes y tabla de uso de IA.



Capturas 06 y 07: inicio e índice del README e instalación (render local del Markdown; conviene revisar también la vista de GitHub).

7. .gitignore y contenido del repositorio

```
node_modules/ dist/ *.tsbuildinfo
.env .env.* !.env.example
npm-debug.log* *.log .DS_Store Thumbs.db
```

Verificación	Resultado
Archivos ignorados correctamente	node_modules/, dist/, .env, .env.local ignorados (<i>git check-ignore</i>); 0 archivos de node_modules versionados
Plantilla de entorno	.env.example versionado (excepción <i>!.env.example</i>)
Código y datos	src/ completo (42 archivos), src/data/ (especialidades y profesionales), data/ (turnos y médicos)
Documentación y configuración	package.json, pacientes-turnos.md, README.md, colección y environment de Postman, informes PDF y capturas
Calidad	npm run lint y npm run format:check sin errores tras los cambios; historial publicado en GitHub (rama main)

8. Uso de Inteligencia Artificial

Herramienta: **Claude Code** (Claude Sonnet 5) trabajando sobre el repositorio. La columna *Ajuste manual* registra lo que aportó quien entrega el trabajo.

Tarea	Herramienta	Prompt (resumen)	Respuesta generada	Ajuste manual
Diseño del módulo Pacientes y Turnos (mockup), <code>{{baseUrl}}</code> en Postman y README integral	Claude Code (Sonnet 5)	"Diseñar las interfaces TypeScript de Paciente y Turno, documentarlas en pacientes-turnos.md con dos endpoints propuestos, reemplazar las URLs fijas de la colección por <code>{{baseUrl}}</code> , re-correr toda la colección y armar un README que documente toda la API leyendo el código real."	Interfaces en src/models validadas con tsc contra los ejemplos JSON; auditoría de la colección (71 requests ya usaban <code>{{baseUrl}}</code> , los ejemplos guardados tenían URL fija); 4 corridas de Newman con distintos baseUrl; README generado con respuestas reales; capturas de evidencia.	Definé los requisitos y el orden de las etapas (commits separados) y pedí generar capturas de pantalla como evidencia.

9. Conclusión y alcance

- El módulo Pacientes y Turnos quedó **diseñado y documentado** (modelo, relaciones, contrato de 2 endpoints y plan de implementación) sin alterar la API vigente; la implementación queda como siguiente etapa.
- La colección de Postman es **independiente del servidor**: cambiar *baseUrl* alcanza para apuntar a otro host o a un Mock Server; 314/314 assertions en verde.
- El README permite a una persona nueva instalar, ejecutar y probar cada endpoint sin leer el código, y el repositorio contiene todo lo necesario para reproducir el proyecto.
- Limitación: las capturas de los documentos Markdown (01–09) son renders locales, no la vista de GitHub.